How Optimizers Shape Representation Geometry in Language Models

Carlo Maggi, Valentin Planes, Giacomo Porpiglia
EPFL - Switzerland

***Abstract*—We study how optimizer choice affects the evolution of representation rank in language models, measured via RankMe, a label-free metric of embedding space utilization. Training a 50M parameter decoder-only transformer with five optimizers (SGD, Adam, AdamW, AdeMAMix, and Muon), we find that Muon and AdamW achieve comparable final RankMe scores, both significantly higher than Adam and AdeMAMix, despite the latter two reaching similar validation loss to AdamW. Interestingly, Muon exhibits a sharp acceleration in both RankMe growth and loss decrease at the onset of the learning rate cooldown, a behavior absent in all other optimizers and whose magnitude scales inversely with the cooldown ratio. These findings suggest that the learning rate cooldown phase plays a significant role in Muon's representational efficiency, and motivate further investigation into the interaction between optimizer dynamics and representation geometry at scale.**

I. Introduction

LLM capabilities emerge from the geometry of learned representations, yet standard metrics like loss are poor predictors of representation quality. RankMe [1] addresses this by measuring the effective rank of the embedding space, providing a label-free metric that correlates with downstream performance. Recent work on Large Language Models (LLMs) [2] shows that RankMe evolves non-trivially during training, and how it correlates with downstream performance and final model quality. The optimizer governs, among all, how parameter updates are distributed across dimensions at each training step, making it a natural candidate for influencing the geometry of the learned representation space. Yet the relationship between optimizer choice and representation rank evolution has not been systematically studied. Muon [3] is a recently proposed optimizer that has rapidly established itself as the de facto standard in recent LLM works. Its use of orthogonalized updates that it may interact differently with the representation space compared to adaptive learning rate methods like Adam. We compare five optimizers (SGD, Adam [4], AdamW [5], AdeMAMix [6] and Muon [3]) on a 50M parameter Language Model, tracking the RankMe score evolution throughout training. We show that Muon yields a RankMe score comparable with AdamW, and higher than all other optimizers. Interestingly, we also observe a clear acceleration in RankMe growth when using Muon coinciding with the start of the learning rate cooldown, a behavior not observed in other optimizers. We further investigate this by testing 3 different learning rate schedules, confirming across all of them that the start of the cooldown phase is the key of Muon's representational efficiency.

II. Methods

A. RankMe

A natural way to assess the quality of learned representations is by measuring how much of the total embedding space is utilized. A model that collapses its representations heavily into a low-dimensional manifold is less likely to capture effectively the richness of the language semantics. RankMe builds on this intuition by providing a measure of effective rank. Given a model with embedding size D, we collect $N \gg D$ activations into a matrix $X \in \mathbb{R}^{N \times D}$. Specifically, we collect the embedding of the last token on the last layer for $N = 1024$ sentences. The final-layer representation of the last token is a natural choice as it constitutes the summary representation in a decoder-only model. We then compute the centered covariance matrix Σ, and normalize its eigenvalues to obtain a probability distribution $p_k = \frac{\lambda_k}{\sum_j \lambda_j}$. RankMe is the exponentiated entropy of p:

$$\text{RankMe}(X) = \exp\left(-\sum_k p_k \log p_k\right)$$

.

During each training, we store checkpoints every 400 update steps, and then compute RankMe for each of them to analyze its evolution.

B. Training

The adopted model architecture is a standard decoder-only transformer with an embedding size of 512, and a depth of 8. More details on the architectural choices are reported in Appendix A. Each model is trained using a dataset of $100,000$ documents from the DCLM-edu dataset [1] for $50,000$ update steps, using an effective batch size of $131,072$ tokens. Regarding the learning rate, we adopt a WSD schedule [7], which consists in a linear increase of the learning rate, a constant phase, and a linear cooldown. We use the same base learning rate value of 3×10^{-4} and a Muon learning rate of 0.02 for all our experiments. To more robustly analyze the importance of the cooldown phase in Muon, we vary its starting point across three configurations: 10%, 30% and 50% of the training steps left. All other hyperparameters are held fixed.

[1] https://huggingface.co/datasets/HuggingFaceTB/dclm-edu

C. Optimizers

We evaluate five optimizers that span a range of update rules and design philosophies: SGD, Adam, AdamW, AdeMAMix, and Muon.

SGD, Adam and AdamW are already well established in the literature. AdeMAMix combines two EMA components with different decay rates, maintaining a slow moving average to better exploit historical gradients.

Muon applies Nesterov momentum combined with the orthogonalization of the update matrix via Newton-Schulz iterations. These steps ensure that the update matrix has roughly the same singular values, ensuring more uniform updates across the weights of 2D matrices. This is the property that motivates our hypothesis that Muon may lead to higher effective rank in the learned representations.

Hyperparameters for all optimizers are reported in Appendix B.

III. Results

A. Training

We first train the model with the mentioned optimizers for $50,000$ update steps. The results are shown in Fig. 1, where we compare the optimizers using 3 different LR-cooldown ratios, as discussed.

For the following observations, we don't include SGD, as its behavior is uncorrelated with the other optimizers, and acts more as a very naive baseline.

The training results show that Muon consistently achieves the lowest validation loss in all three scenarios. Interestingly, we also observe how, before the cooldown phase starts, Muon always has a higher validation loss compared to the other optimizers. When the cooldown phase starts, the slope of the loss decreases abruptly, yielding a much faster decrease compared to the other optimizers, for which the decrease ratio stays roughly constant. We also note that this effect is amplified if the cooldown value is smaller. These observations are made explicit in table I, where we compute the slope of the validation loss decrease in the cooldown phase.

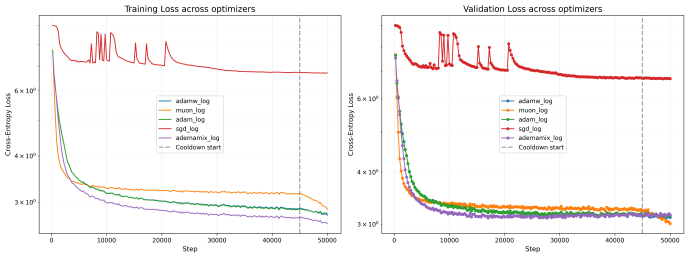
cooldown (CD)	0.1	0.3	0.5
Validation loss at CD start	3.231	3.2668	3.2921
Final validation loss	3.0094	3.0006	3.0018
Slope after CD start (10^{-5})	-4.494	-1.7747	-1.1612
Slope before CD start (10^{-5})	-0.766	-0.206	-0.026

TABLE I: Validation loss values and slope across different cooldown ratios with Muon.

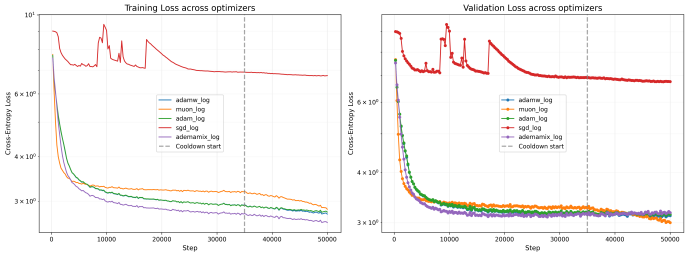
B. RankMe analysis

Next, we compute the RankMe score for each of the checkpoints saved during training with each optimizer. The results are shown in Fig. 2

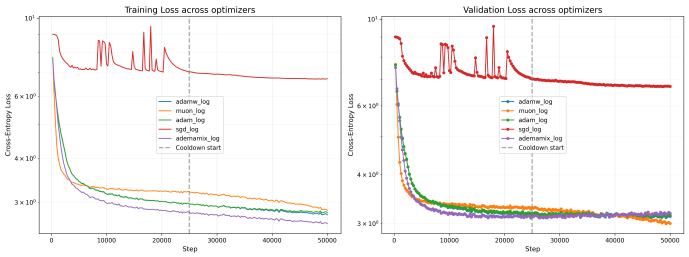
The results show how AdamW and Muon consistently achieve very similar RankMe values at the end of training, indicating that they yield a similar utilization of the embedding space. Adam and AdaMAMix always produce more collapsed representations with fewer effective dimensions per token.



(a) Training for cooldown ratio $= 0.1$



(b) Training for cooldown ratio $= 0.3$



(c) Training for cooldown ratio $= 0.5$

Fig. 1: Training dynamics for different cooldown ratios.

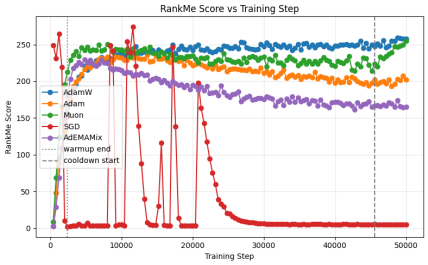
This is an interesting finding, as the differences in RankMe are not reflected by the validation loss: despite having similar RankMe at the end of training, Muon achieves a significantly lower validation loss; at the same time, AdeMAMix and Adam achieve a similar loss compared to AdamW, while having a significantly lower RankMe value.

Similarly to what we observe for the validation loss, what distinguishes Muon from other optimizers including AdamW is an abrupt increase in the RankMe slope at the cooldown starting point, more pronounced for smaller cooldown ratios. Notably, before cooldown the slope is slightly negative, indicating that representations are slowly collapsing into a lower-dimensional manifold; the onset of cooldown sharply reverses this trend, with the magnitude of the recovery scaling inversely with the cooldown ratio. In table II we report the quantitative results.

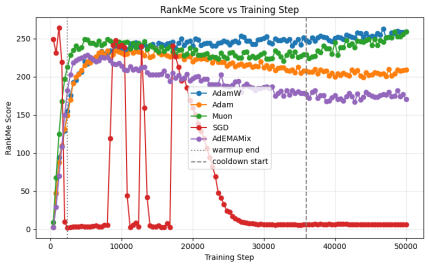
The slopes in both table I and II were computed as

$$\text{slope} = \frac{Final\_value - Initial\_value}{total\_steps \cdot CD}$$

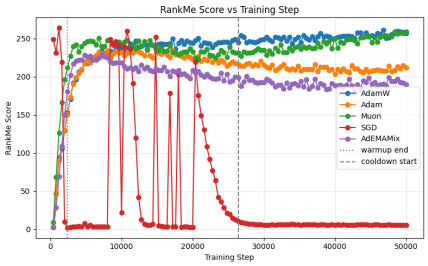
Note that in both table I and II the initial values are taken $5,000$ steps before the start of the CD, while the final values are taken at the final step, i.e., $50,000$.



(a) RankMe score across all optimizers with cooldown $= 0.1$



(b) RankMe score across all optimizers with cooldown $= 0.3$



(c) RankMe score across all optimizers with cooldown $= 0.5$

Fig. 2: RankMe evolution across all optimizers for different cooldown ratios.

cooldown (CD)	0.1	0.3	0.5
RankMe value at CD start	218.0905	226.3783	232.1015
Final RankMe value	255.088	258.6690	259.1462
Slope after CD start (10^{-3})	7.3995	2.1527	1.0818
Slope before CD start (10^{-3})	-0.5496	-0.8921	-0.08454

TABLE II: RankMe values and slope across different cooldown ratios with Muon.

IV. Discussion

A. RankMe comparison

The results show that the use of the Muon optimizer does not yield a more uniform use of the embedding space dimensions. Instead, the RankMe values at the end of training are very similar to the ones obtained using AdamW on all model parameters. Adam and AdeMAMix show significantly worse RankMe scores compared to AdamW and Muon, producing more collapsed representations that use a smaller fraction of the embedding space when compared to AdamW and Muon; nevertheless, their validation loss is very similar to the one obtained by AdamW. This constitutes evidence that RankMe and validation loss can decouple substantially under certain optimizer choices.

B. Cooldown effect on RankMe for Muon

We observe an interesting behavior using Muon during the cooldown phase: the RankMe curve suddenly steepens, closing the gap with the RankMe scores obtained using AdamW. While the same learning rate cooldown is applied to all other optimizers as well, Muon is the only one that shows this behavior, reflecting both in the validation loss and RankMe evolution. The mechanism underlying this interaction between learning rate annealing and Muon's orthogonal update rule remains an open question.

C. Limitations

Due to compute limitations, we couldn't perform a full hyperparameter search to find the optimal learning rates for each optimizer in our specific setting, and instead relied on common-sense values (Table IV, appendix). Therefore, we can't exclude that the steepening we observe could be due to a too high base learning rate for Muon, which causes the observed metrics (both validation loss and RankMe) to change more effectively only once the learning rate is decreased.
Our results are obtained using small-scale language models, with roughly 50 million parameters, on a much smaller scale compared to modern standards. Whether our findings also extend to large-scale LLMs requires further investigation.

V. Summary

In this work, we explored how different optimizer choices affect the representation rank in small-scale language models. We find that Muon and AdamW yield very similar RankMe scores, significantly higher than the other tested optimizers. Muon shows a unique acceleration in both validation loss and RankMe at the start of the cooldown phase. This may suggest that the cooldown choice is much more critical for Muon's representational efficiency than for other optimizers. Whether these observations scale to larger models, and whether the cooldown interaction is optimizer-specific or a hyperparameter artifact, awaits further investigation.

References

[1] Q. Garrido, R. Balestriero, L. Najman, and Y. Lecun, "Rankme: Assessing the downstream performance of pretrained self-supervised representations by their rank," 2023.
[2] M. Z. Li, K. K. Agrawal, A. Ghosh, K. K. Teru, A. Santoro, G. Lajoie, and B. A. Richards, "Tracing the representation geometry of language models from pretraining to post-training," 2025.
[3] J. Liu, J. Su, X. Yao, Z. Jiang, G. Lai, Y. Du, Y. Qin, W. Xu, E. Lu, J. Yan, Y. Chen, H. Zheng, Y. Liu, S. Liu, B. Yin, W. He, H. Zhu, Y. Wang, J. Wang, M. Dong, Z. Zhang, Y. Kang, H. Zhang, X. Xu, Y. Zhang, Y. Wu, X. Zhou, and Z. Yang, "Muon is scalable for llm training," 2025.
[4] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," 2017.
[5] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2019.
[6] M. Pagliardini, P. Ablin, and D. Grangier, "The ademamix optimizer: Better, faster, older," 2024.

[7] S. Hu, Y. Tu, X. Han, C. He, G. Cui, X. Long, Z. Zheng, Y. Fang, Y. Huang, W. Zhao, X. Zhang, Z. L. Thai, K. Zhang, C. Wang, Y. Yao, C. Zhao, J. Zhou, J. Cai, Z. Zhai, N. Ding, C. Jia, G. Zeng, D. Li, Z. Liu, and M. Sun, "Minicpm: Unveiling the potential of small language models with scalable training strategies," 2024.
[8] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," 2023.
[9] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, and Y. Liu, "Roformer: Enhanced transformer with rotary position embedding," 2023.

APPENDIX

A. Training configurations

We train a GPT-style decoder-only transformer. The architecture follows the autoresearch codebase [2], which includes several modern modifications on top of the original transformer architecture proposed in [8]. Table III summarises the configuration.

TABLE III: Model architecture hyperparameters.

Hyperparameter	**Value**	**Notes**
Number of layers	8	Transformer blocks
Hidden dimension	512	depth $\times$ aspect ratio (8×64)
Attention heads	8	
Head dimension	64	
KV heads	8	Standard MHA (no GQA)
MLP hidden dim	2048	$4\times$ hidden dimension
Context length	1024	tokens
Vocabulary size	8192	BPE tokenizer trained from scratch
Total parameters	50.3M	
Architecture features		
Position encoding	RoPE	[9]
Normalization	RMSNorm	pre-norm, no bias
MLP activation	ReLU^2	squared ReLU
Attention normalization	QK-Norm	applied after RoPE
Value embeddings	ResFormer-style	alternating layers
Residual scaling	per-layer λ_{resid}, λ_0	learnable scalars
Logit soft-capping	cap $= 15$	$\hat{l} = 15\tanh(l/15)$
Precision	bfloat16	

B. Optimizer configurations

All optimizers share the same learning rate schedule: linear warmup over the first 5% of steps, constant phase, then linear cooldown over the last 50% of steps to 10% of the peak learning rate.

Table IV summarises the hyperparameters for each optimizer. For Muon, 2-D hidden weight matrices use the Muon update rule; all remaining parameters (embeddings, LM head, 1-D scalars) fall back to AdamW.

TABLE IV: Optimizer hyperparameters. Dashes indicate the parameter is not applicable. AdEMAMix β_3 and α are warmed up linearly from 0 over the first 10 000 steps.

	AdamW	**Adam**	**SGD**	**Muon**	**AdEMAMix**
Peak LR	3×10^{-4}	3×10^{-4}	3×10^{-4}	0.02 / 3×10^{-4}	3×10^{-4}
β_1	0.9	0.9	—	0.95 / 0.9	0.9
β_2	0.95	0.95	—	— / 0.95	0.999
β_3	—	—	—	—	0.999
α	—	—	—	—	5
Momentum	—	—	0.9	—	—
Nesterov	—	—	yes	—	—
Weight decay	0.1	0	0.1	0.1	0.1

[2]https://github.com/karpathy/autoresearch